unit2-image-enhancement-notes-v2

Unit 2: Image Enhancement (Spatial Domain) - Complete Study Notes (v2)

Course: Image and Video Processing (IVP)
Level: SVKM's NMIMS MPSTME, Semester V (B.Tech COMP/EXTC)
Primary Source: `IVP UNIT 2.pptx` (and official course policy documents)
Syllabus Covered: 1. **Point Processing Operations**: Digital Negative/Complement, Contrast Stretching, Global Thresholding, Intensity-Level Slicing, Logarithmic Transformation, Power-Law (Gamma) Transformation. 2. **Histogram Processing**: Histogram Equalization (HE), HE Algorithm/Procedure, and Solved Numericals. 3. **Neighborhood Operations / Spatial Filters (SMOOTHING ONLY)**: Low-Pass Filtering, Average Filter, Weighted Average Filter, Median Filter, Gaussian Filtering.

📖 SECTION 1: Introduction to Spatial Domain Image Enhancement

1. What is Spatial Domain Image Enhancement?

- **Definition:** Image enhancement refers to the process of manipulating a digital image so that the result is more suitable than the original image for a specific application [316]. The term **"Spatial Domain"** denotes the physical 2-D pixel grid of the image [315].
- **Mathematical Model:** Spatial domain processing operates directly on the pixels of an image [37]. It is represented mathematically by the general expression:

  $$g(x,y) = T[f(x,y)]$$

  [175] where:
  - $f(x,y)$ is the input image [6].
  - $g(x,y)$ is the processed output image [175].
  - T is an operator or transformation function defined over a specified neighborhood of coordinate (x,y) [175].

2. Point Processing vs. Neighborhood Processing

- **Point Processing (Pixel-by-Pixel):** The neighborhood around pixel (x,y) has a size of 1×1 [83]. The output intensity s at any coordinate depends *only* on the input intensity r at that exact coordinate [83].
  - *Equation:* $s = T(r)$ [83, 175]
- **Neighborhood Processing (Filtering):** The neighborhood is larger (e.g., 3×3 or 5×5) [84]. The output intensity $g(x,y)$ is computed based on both the center pixel and its surrounding neighbors [84].

⭐ **Must Remember:** Enhancement is a **subjective process** [53]. There is no general "universal" theory of image enhancement [316]. What is "good" enhancement for a medical radiologist (highlighting bones) might be "poor" enhancement for a computer vision navigation system [294].

📐 SECTION 2: Point Processing Operations

Point operations map an input intensity level r to an output intensity level s [83]. For an 8-bit image, intensity levels range from $[0, L-1]$ where $L = 2^8 = 256$ [41, 79].

1. Digital Negative / Complement

- **Definition:** A transformation that subtracts each pixel value from the maximum possible intensity level of the image, reversing its gray-level scale [66, 117].
- **Basic Concept and Intuition:** Maps bright values to dark ones and dark values to bright ones, producing the digital equivalent of a photographic negative [117, 159]. This is highly intuitive for visual verification—bone details in radiographs appear as dark lines on a bright background.
- **Mathematical Formula:**

  $$s = (L-1) - r$$

  [66]
- **Variable Meanings:**
  - r: Input pixel intensity level ($r \in [0, L-1]$) [66].
  - s: Transformed output pixel intensity level [66].
  - L: Total number of gray levels in the image ($L = 2^k$ for a k-bit image) [31, 91].
  - $L-1$: Maximum possible intensity value on that scale [31, 91].
- **Step-by-Step Method:**
  1. Identify the bit depth k of the image [31].
  2. Compute the maximum gray level $L-1 = 2^k - 1$ [31].
  3. Subtract each input pixel's value r from $L-1$ to compute s [31].
  4. Construct the output matrix using the computed s values.
- **Example (3-bit, $L-1=7$):** For input $r = 2 \implies s = 7 - 2 = 5$. For input matrix A in Section 2:

  $$A_{\text{neg}} = \begin{bmatrix} 6 & 5 & 5 & 5 & 5 \\ 4 & 5 & 3 & 2 & 5 \\ 0 & 0 & 0 & 0 & 1 \\ 5 & 5 & 5 & 4 & 4 \\ 3 & 3 & 3 & 3 & 3 \end{bmatrix}$$

- **Applications:** Medical imaging (e.g., chest X-rays, mammograms) to show small details in dominant dark areas more clearly [74, 117].
- **Advantages:** Simple arithmetic subtraction; requires zero training or parameter tuning; highly effective for clinical analysis of dense tissues [74, 117].
- **Limitations:** Completely washes out contrast in bright regions; unsuitable for standard, well-illuminated digital photographs.
- ⭐ **Must Remember:** Always use $L-1$ as the subtrahend, not L. Subtracting from 256 instead of 255 results in out-of-range pixels!
- 🧠 **Must Understand:** It is a linear transformation with a negative slope ($\text{slope} = -1$) and an intercept of $L-1$.
- ✍️ **Exam Focus:** Verify the bit depth first! A "3-bit image negative" means subtracting from 7, not 255.
- ⚠️ **Common Mistakes:** Forgetting to subtract from $L-1$ and using L instead, resulting in overflow.

---

2. Contrast Stretching

- **Definition:** A piecewise-linear transformation designed to expand the dynamic range of gray levels in an image [122].
- **Basic Concept and Intuition:** Low-contrast images often result from poor illumination or sensor constraints, clustering the histogram in a tight band. Contrast stretching "pulls" this clustered band apart, stretching it across the full display scale ($[0, L-1]$) to increase the perceptual contrast.
- **Mathematical Formula (Piecewise Linear):**

  $$s = \begin{cases} \alpha \cdot r & 0 \le r < r_1 \\ \beta \cdot (r - r_1) + s_1 & r_1 \le r < r_2 \\ \gamma \cdot (r - r_2) + s_2 & r_2 \le r \le L-1 \end{cases}$$

  [281] Where regional slopes are calculated as:

  $$\alpha = \frac{s_1}{r_1}, \quad \beta = \frac{s_2 - s_1}{r_2 - r_1}, \quad \gamma = \frac{(L-1) - s_2}{(L-1) - r_2}$$

  [281]
- **Variable Meanings:**
  - r, s: Input and output pixel intensities.
  - (r_1, s_1) and (r_2, s_2): Control points that define the boundaries and slopes of the three segments [281].
  - α, β, γ: Slopes of the respective segments [281].
  - $L-1$: Maximum possible intensity value.
- **Step-by-Step Method:**
  1. Identify control points (r_1, s_1) and (r_2, s_2) from the problem statement [281].
  2. Compute regional slopes α, β, and γ [281].
  3. For each pixel r, check which region it falls into and apply the corresponding formula [183].
  4. Round the resulting s value to the nearest integer.
  5. Construct the output matrix.
- **Example (3-bit, $L-1=7$):** For $(r_1, s_1) = (3, 2)$ and $(r_2, s_2) = (5, 6)$, the slopes are $\alpha = 0.67$, $\beta = 2.0$, $\gamma = 0.5$. The mapped matrix is:

  $$A_{\text{stretched}} = \begin{bmatrix} 4 & 2 & 6 & 1 \\ 2 & 7 & 4 & 7 \\ 1 & 1 & 7 & 6 \\ 7 & 7 & 4 & 1 \end{bmatrix}$$

- **Applications:** Contrast enhancement of satellite, aerial runway, or electron microscope imagery [122].
- **Advantages:** Flexible and tunable contrast adjustment by shifting control points; simple linear calculations.
- **Limitations:** Requires manual selection of control points; can introduce harsh threshold-like binarization if slopes are overly steep.
- ⭐ **Must Remember:** If slopes are > 1, contrast is stretched; if < 1, contrast is compressed; if $= 1$, it remains unchanged (identity).
- 🧠 **Must Understand:** When $r_1 = r_2$ and $s_1 = 0, s_2 = L-1$, the transformation simplifies to a **thresholding function**, producing a binary image.
- ✍️ **Exam Focus:** Show intermediate slope calculations and mapped intensity tables clearly.
- ⚠️ **Common Mistakes:** Misclassifying boundary pixels (e.g. applying the wrong segment formula at $r = r_1$ or $r = r_2$).

---

3. Global Thresholding

- **Definition:** A point operation that binarizes an image based on a single global threshold value T, segmenting it into foreground and background [19].
- **Basic Concept and Intuition:** Separates an image with a bimodal histogram into two regions. Any pixel brighter than T becomes white, and any pixel darker than or equal to T becomes black.
- **Mathematical Formula:**

  $$s = \begin{cases} L-1 & \text{if } r > T \\ 0 & \text{if } r \le T \end{cases}$$

  [94, 290]
- **Variable Meanings:**
  - r, s: Input and output intensities.
  - T: The binarization threshold.
  - $L-1$: Maximum intensity (white).
- **Step-by-Step Method (Iterative Estimation):**
  1. Compute an initial threshold T_0 as the average intensity of all pixels in the image [95, 265].
  2. Segment the image using T_0 to form two classes of pixels: G_1 (pixels $> T_0$) and G_2 (pixels $\le T_0$) [95, 96, 265].
  3. Calculate the mean intensities μ_1 and μ_2 for G_1 and G_2 [96, 265].
  4. Update the threshold: $T_{\text{new}} = \frac{\mu_1 + \mu_2}{2}$ [96, 270].
  5. Repeat Steps 2-4 until the threshold converges ($T_{\text{new}} = T_{\text{previous}}$) [96, 271].
- **Example (3×3 matrix):**

  $$f(x,y) = \begin{bmatrix} 5 & 3 & 9 \\ 2 & 1 & 7 \\ 8 & 4 & 2 \end{bmatrix}$$

  Iterative global thresholding converges at $T = 6$, yielding:

  $$g(x,y) = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

- **Applications:** Document binarization, optical character recognition (OCR) preprocessing, segmenting distinct high-contrast objects.
- **Advantages:** Extremely fast, fully automated, requires no human intervention for bimodal histograms.
- **Limitations:** Fails completely under non-uniform illumination or shadows.
- ⭐ **Must Remember:** The partition groups G_1 and G_2 must contain actual pixel intensity values, while means μ_1, μ_2 are arithmetic averages of those values.
- 🧠 **Must Understand:** Convergence is guaranteed when the split groups remain identical between iterations.
- ✍️ **Exam Focus:** Write down the elements of G_1 and G_2 at each iteration and compute class means carefully.
- ⚠️ **Common Mistakes:** Miscalculating the class means or neglecting to round according to specified rules.

---

4. Intensity-Level Slicing / Gray-Level Slicing

- **Definition:** An intensity transformation that highlights a specific range of gray levels $[a, b]$ of interest [90, 123].
- **Basic Concept and Intuition:** Similar to a bandpass filter. If we are interested in water bodies or specific blood vessel densities, we highlight that specific intensity band $[a, b]$ as white, while either setting all other pixels to black or leaving them unchanged.
- **Mathematical Formulas:**
  1. **Without Background Preservation (Binary Slicing):**

     $$s = \begin{cases} L-1 & \text{if } a \le r \le b \\ 0 & \text{otherwise} \end{cases}$$

     [64, 275]
  2. **With Background Preservation:**

     $$s = \begin{cases} L-1 & \text{if } a \le r \le b \\ r & \text{otherwise} \end{cases}$$

     [198, 275]
- **Variable Meanings:**
  - r, s: Input and output pixel values.
  - $[a, b]$: Selected gray-level band of interest.
  - $L-1$: Highlight intensity value (usually max gray level).
- **Step-by-Step Method:**
  1. Determine the highlight range $[a, b]$ (often specified as percentages of the max possible intensity).
  2. Check each pixel's intensity r. If it lies inside $[a, b]$, map it to $L-1$.
  3. If r lies outside $[a, b]$:
     - Map to 0 (Without Background).
     - Keep original value r (With Background).
  4. Construct the output matrix.
- **Example (3-bit, target $r \in [3, 4]$):** For matrix A in Section 2,

  $$A_{\text{without}} = \begin{bmatrix} 0 & 7 & 0 & 0 & 7 \\ 0 & 7 & 7 & 0 & 0 \\ 0 & 7 & 0 & 0 & 0 \\ 7 & 0 & 7 & 0 & 0 \\ 0 & 7 & 0 & 7 & 0 \end{bmatrix}$$

- **Applications:** Enhancing specific geographic elevations or water bodies in satellite imaging; highlighting flaws in industrial X-rays [181].
- **Advantages:** Outstanding for targeted feature isolation.
- **Limitations:** Slicing without background discards all other contextual visual details.
- ⭐ **Must Remember:** Slicing with background keeps other pixels unchanged as r, while slicing without background sets them to 0.
- 🧠 **Must Understand:** It is a piecewise constant point operation.
- ✍️ **Exam Focus:** Be careful when the range is described using percentages (e.g. "40% to 70% of max intensity").
- ⚠️ **Common Mistakes:** Accidentally setting the background pixels to 0 when "With Background" is requested, or vice-versa.

---

5. Logarithmic Transformation

- **Definition:** A non-linear point operation that maps a narrow range of low-intensity dark values into a wider range of output levels [85, 118, 177].
- **Basic Concept and Intuition:** Human visual perception is roughly logarithmic. When an image contains a massive range of intensities, high values dominate the display, leaving dark details completely invisible. The log transform compresses this range, expanding low gray levels while compressing high ones.
- **Mathematical Formula:**

  $$s = c \log_{10}(1 + r)$$

  [41, 65, 256]
- **Variable Meanings:**
  - r, s: Input and output pixel values ($r \ge 0$).
  - c: Scaling constant used to stretch output values across the display range [277].
  - $\log_{10}$: Base-10 logarithm.
- **Step-by-Step Method:**
  1. Identify or calculate c using: $c = \frac{L-1}{\log_{10}(1+(L-1))}$ [277].
  2. Compute the transformed value $s = c \log_{10}(1 + r)$ for each unique intensity level.
  3. Round computed output values to the nearest integer.
  4. Reconstruct the output matrix.
- **Example (3-bit, $c = 8$):**

  $$A = \begin{bmatrix} 2 & 3 & 0 & 6 & 7 \\ 0 & 3 & 7 & 5 & 2 \\ 5 & 3 & 2 & 4 & 0 \\ 4 & 2 & 2 & 1 & 0 \\ 1 & 7 & 6 & 4 & 5 \end{bmatrix} \implies A_{\log} = \begin{bmatrix} 4 & 5 & 0 & 7 & 7 \\ 0 & 5 & 7 & 6 & 4 \\ 6 & 5 & 4 & 6 & 0 \\ 6 & 4 & 4 & 2 & 0 \\ 2 & 7 & 7 & 6 & 6 \end{bmatrix}$$

- **Applications:** Compressing the high dynamic range of Fourier spectrum coefficients so low-intensity spectrum details are visible [76].
- **Advantages:** Dramatically enhances dark details without losing context.
- **Limitations:** Compresses and washes out contrast in bright highlight regions; shape of the curve is fixed.
- ⭐ **Must Remember:** We add 1 to r because $\log(0)$ is mathematically undefined! Adding 1 maps $0 \to \log(1) = 0$.
- 🧠 **Must Understand:** Base-10 logarithms ($\log_{10}$) are standard in NMIMS exam matrices, though natural log ($\ln$) is also used in some textbook formulas.
- ✍️ **Exam Focus:** Show your mapping lookup table for values 0 to 7 clearly before applying it to the matrix.
- ⚠️ **Common Mistakes:** Forgetting to add 1 to r, leading to calculation errors at $r = 0$.

---

6. Power-Law / Gamma Transformation

- **Definition:** A non-linear point operation that raises the input intensity to a power γ, used for contrast tuning and display calibration [13, 201].
- **Basic Concept and Intuition:** Most display devices, camera sensors, and printers exhibit a non-linear relationship between voltage and brightness (known as the device gamma). Power-law transforms are used to pre-compensate (Gamma Correction) to ensure realistic linear visual display.
- **Mathematical Formula:**

  $$s = c \cdot r^{\gamma}$$

  [43, 65, 231]
- **Variable Meanings:**
  - r, s: Input and output pixel intensities.
  - c, γ: Positive constants [43, 231].
  - γ: The power exponent (gamma) [27].
- **Step-by-Step Method:**
  1. Determine parameters c and γ.
  2. Map each level using $s = c \cdot r^{\gamma}$ [10].
  3. Clip values at the maximum allowed gray level $L-1$ if direct non-normalized exponentiation exceeds the range.
  4. Round mapped values to the nearest integer.
  5. Construct the output matrix.
- **Example (3-bit, $c = 3, \gamma = 0.5$):**

  $$A_{\text{gamma}} = \begin{bmatrix} 4 & 5 & 0 & 7 & 7 \\ 0 & 5 & 7 & 7 & 4 \\ 7 & 5 & 4 & 6 & 0 \\ 6 & 4 & 4 & 3 & 0 \\ 3 & 7 & 7 & 6 & 7 \end{bmatrix}$$

- **Applications:** Gamma correction for display calibration, detail expansion in washed-out aerial images or dark medical scans [230].
- **Advantages:** Highly versatile; a family of curves is generated simply by varying γ [114].
- **Limitations:** Over-enhancement can introduce noise; requires proper normalization to avoid numerical overflows.
- ⭐ **Must Remember:** $\gamma < 1$ stretches dark levels (similar to log); $\gamma > 1$ stretches bright levels (darkens the image).
- 🧠 **Must Understand:** Standard computer displays assume a target gamma of 2.2, requiring a correction exponent of $1/2.2 \approx 0.45$.
- ✍️ **Exam Focus:** Note whether the question asks for normalized intensities or direct raw integer calculations.
- ⚠️ **Common Mistakes:** Misapplying fractional powers (e.g. squaring instead of taking the square root for $\gamma = 0.5$).

📊 SECTION 3: Histogram Processing

1. What is an Image Histogram?

A histogram represents the frequency of occurrence of each gray level in an image [182]. For an image with gray levels in range $[0, L-1]$, the histogram is defined as:

$$h(r_k) = n_k$$

where: * r_k: The k-th gray level [182]. * n_k: The number of pixels having intensity r_k [182]. * The **normalized histogram** is the Probability Density Function (PDF): $p_r(r_k) = \frac{n_k}{N}$, representing the likelihood of occurrence of level r_k [19].

---

2. Histogram Equalization (HE)

- **Definition:** A contrast enhancement technique that achieves a more uniform intensity distribution by mapping the cumulative distribution function (CDF) of the image's histogram to the output gray levels [182].
- **Basic Concept and Intuition:** Spreads out the most frequent intensity levels across the entire gray-level spectrum, effectively flattening the histogram and maximizing visual contrast.
- **Transformation Function:**

  $$s_k = T(r_k) = (L-1) \cdot \sum_{j=0}^{k} p_r(r_j) = (L-1) \cdot \text{CDF}(r_k)$$

  where:
  - $p_r(r_j) = \frac{n_j}{N}$ is the PDF.
  - N is the total number of pixels in the image [182].
  - $\text{CDF}(r_k)$ is the Cumulative Distribution Function [182].
- **Step-by-Step Algorithm (The Exact Procedure):**
  1. Count the pixel frequency n_k for each gray level r_k from 0 to $L-1$.
  2. Sum the frequencies to find the total pixel count $N = \sum n_k$.
  3. Compute the Probability Density Function (PDF) for each level: $p_r(r_k) = \frac{n_k}{N}$.
  4. Compute the running sum of PDFs to find the Cumulative Distribution Function (CDF): $\text{CDF}(r_k) = \sum_{j=0}^{k} p_r(r_j)$.
  5. Calculate the transformed scaled values: $s'_k = (L-1) \cdot \text{CDF}(r_k)$.
  6. Round the scaled values to the nearest integer: $s_k = \text{round}(s'_k)$.
  7. Remap the original pixel values in the input image matrix using the mapped values s_k.
  8. Construct the output matrix and count the final distribution.

---

3. Solved Numerical Example

(Refer to the detailed calculation table and output matrix mapping shown in the Section 3 HE numerical above, which maps input matrix f to output g):

$$f(x,y) = \begin{bmatrix} 5 & 2 & 2 \\ 6 & 7 & 3 \\ 3 & 7 & 3 \end{bmatrix} \quad \longrightarrow \quad g(x,y) = \begin{bmatrix} 5 & 2 & 2 \\ 5 & 7 & 4 \\ 4 & 7 & 4 \end{bmatrix}$$

* **Applications:** Medical scans (X-rays, MRIs), satellite imaging, underwater imaging, and low-contrast consumer photos. * **Advantages:** Fully automated and parameter-free; highly consistent mathematical Point operation. * **Limitations:** Can over-enhance background noise; is not suitable for color images if applied to RGB channels directly (causes severe color distortion; must use YCbCr/HSV luminance equalization instead). * ⭐ **Must Remember:** Sum of all normalized frequencies (PDF) must equal 1.0, and the final CDF value must equal 1.0. * 🧠 **Must Understand (Why HE is not perfectly flat):** Remapping discrete histogram components cannot split pixel clusters. Since it is a point transform, all pixels with input value r_k must map together to output value s_k. Therefore, discrete HE only redistributes components but cannot produce a perfectly uniform, flat histogram [94]. * ✍️ **Exam Focus:** Set up your table columns exactly as shown in the PPT: Gray level $\to n_k \to \text{PDF} \to \text{CDF} \to (L-1) \cdot \text{CDF} \to \text{Rounded } s_k$. * ⚠️ **Common Mistakes:** Skipping the rounding step or forgetting to multiply by the scale factor $L-1$.

🛡️ SECTION 4: Neighborhood Processing & Spatial Filters (SMOOTHING ONLY)

Neighborhood operations (filtering) modify the intensity of a pixel based on the intensities of surrounding pixels inside a small local sub-grid (often 3×3 or 5×5) [84].

```
   3x3 Neighborhood Window
  ┌───┬───┬───┐
  │w1 │w2 │w3 │
  ├───┼───┼───┤
  │w4 │w5 │w6 │   ◄─── Mask / Kernel Coefficients
  ├───┼───┼───┤
  │w7 │w8 │w9 │
  └───┴───┴───┘
```

1. Mechanics of Neighborhood Processing (Spatial Convolution)

The filtered value at center pixel is computed by multiplying mask coefficients with underlying pixel values, and summing them up:

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) \cdot f(x+s, y+t)$$

---

2. Smoothing Spatial Filters (Low-Pass Filtering)

Smoothing is used to reduce high-frequency noise and eliminate irrelevant details, acting as a low-pass filter in the spatial domain [174, 235].

A. Standard Average (Box) Filter

- **Definition:** A linear smoothing filter where the output pixel is the simple arithmetic mean of all pixels in its neighborhood window [83, 174].
- **Mask (3x3):** All coefficients are positive and equal to $1/9$:

  $$H_{\text{box}} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

  [46, 125]
- **Variable Meanings:** Coefficients are $1/(M \cdot N)$ where $M \times N$ is mask size.
- **Step-by-Step Method:**
  1. Place mask over target pixel (x,y).
  2. Sum all pixels covered by the mask.
  3. Divide by the total number of pixels in the window (e.g., 9) [45].
  4. Set this average as the new intensity of the center pixel.
- **Example (Center Pixel calculation):** For sub-matrix centered at 15:

  $$\text{Neighborhood} = \begin{bmatrix} 3 & 9 & 11 \\ 7 & 15 & 8 \\ 10 & 12 & 9 \end{bmatrix} \implies \text{Average} = \frac{84}{9} = \mathbf{9.33}$$

  [125]
- **Applications:** Noise reduction, general blurring to remove fine details.
- **Advantages:** Extremely simple to implement; very fast computation.
- **Limitations:** Blurs sharp edges significantly, losing important structural details.
- ⭐ **Must Remember:** All mask coefficients must be positive and normalize to sum to 1 to preserve overall image brightness [36].
- 🧠 **Must Understand:** As window size increases, the blurring effect becomes stronger and more details are lost [39].
- ✍️ **Exam Focus:** Divide the final sum by the total number of coefficients (e.g. 9 or 25) to normalize correctly.
- ⚠️ **Common Mistakes:** Forgetting to divide by the normalization constant.

B. Weighted Average Filter

- **Definition:** A linear smoothing filter where pixels closer to the mask center are given higher weights [125, 174].
- **Mask (3x3):** Center is weighted highest, dropping off inversely as a function of distance:

  $$H_{\text{weighted}} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Step-by-Step Method:**
  1. Place mask over target pixel.
  2. Multiply each underlying pixel by its corresponding mask weight.
  3. Sum all weighted products.
  4. Divide by the sum of all mask weights (e.g., 16).
- **Example:** For the sub-matrix above centered at 15:

  $$\text{Sum} = 3(1) + 9(2) + 11(1) + 7(2) + 15(4) + 8(2) + 10(1) + 12(2) + 9(1) = 165 \implies \text{WeightedAverage} = \frac{165}{16}$$

- **Applications:** Smoothing noise while maintaining center-pixel dominance.
- **Advantages:** Blurs sharp edges less than a standard box filter of the same size.
- **Limitations:** Edge blurring is still present; slightly more computational overhead.
- ⭐ **Must Remember:** Weight values must sum to the normalization constant (denominator) to avoid brightness shifts.
- 🧠 **Must Understand:** Giving more weight to the center pixel gives it more "importance" in the local average calculation.
- ✍️ **Exam Focus:** Divide by 16 for a standard 3×3 weighted average mask.
- ⚠️ **Common Mistakes:** Miscalculating the divisor (using 9 instead of 16).

C. Gaussian Filtering

- **Definition:** A linear smoothing filter where the mask coefficients are sampled from a radially symmetric 2-D Gaussian distribution [4, 24].
- **Radically Symmetric Gaussian Function:**

  $$h(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

  [66]
- **Variable Meanings:**
  - σ: Standard deviation, which controls the width/spread of the bell curve (blur degree) [38].
  - x, y: Coordinate distances from the mask center.
- **Step-by-Step Method:**
  1. Sample the continuous 2-D Gaussian function to generate mask coefficients [24].
  2. Normalize the coefficients so they sum to 1 [36].
  3. Convolve the mask with the image.
- **Applications:** Optimal low-pass filtering, high-quality blurring, pre-processing step for Canny edge detection [27, 41].
- **Advantages:** "Well-behaved" in both spatial and frequency domains [25]; **zero overshoot** to step inputs; **completely eliminates ringing artifacts** [60]. It is **separable**, meaning a 2D Gaussian can be computed as two sequential 1D convolutions, making it highly efficient [10].
- **Limitations:** Slightly more complex to compute than box blurs.
- ⭐ **Must Remember:** The degree of blur is controlled by standard deviation σ, not by mask size [25].
- 🧠 **Must Understand:** A 2D Gaussian filter is separable into a pair of 1D Gaussian filters, which reduces complexity from $O(M^2)$ to $O(2M)$ [10, 244].
- ✍️ **Exam Focus:** State the separability property and explain its computational advantage.
- ⚠️ **Common Mistakes:** Setting σ too large or too small without adjusting the mask size.

D. Median Filter (Non-linear Order-Statistic Filter)

- **Definition:** A non-linear spatial filter that replaces the center pixel value with the median of all pixel intensities in its neighborhood window [20, 161].
- **Step-by-Step Method:**
  1. Extract all pixel intensities inside the $M \times N$ neighborhood window [308].
  2. Sort these values in ascending (or descending) order [308].
  3. Identify the middle value of the sorted list (the median) [308].
  4. Replace the original center pixel value with this median [308].
- **Example (3x3 window, center pixel 30):**

  $$\text{Neighborhood} = \begin{bmatrix} 8 & 17 & 4 \\ 3 & 30 & 6 \\ 15 & 10 & 7 \end{bmatrix}$$

  Sorted list: `[3, 4, 6, 7, 8, 10, 15, 17, 30]`. Median = 8. Output replaces 30 with **8**.
- **Applications:** Outstanding for removing **Salt and Pepper noise** (impulsive binary noise) [197, 247].
- **Advantages:** Removes impulse noise with **no edge blurring**, preserving sharp boundaries [247, 292]. Unlike average filters, it never creates new gray levels not present in the original neighborhood [249].
- **Limitations:** Non-linear (cannot be analyzed in frequency domain); computationally slower due to sorting overhead; fails if noise occupies $> 50\%$ of the window area.
- ⭐ **Must Remember:** Mask dimensions must be odd to ensure a unique single center pixel and unique median index.
- 🧠 **Must Understand:** It does not blend or average pixel values; it simply selects an existing value from the neighborhood.
- ✍️ **Exam Focus:** Show the sorted array of 9 pixels clearly before picking the 5th element!
- ⚠️ **Common Mistakes:** Selecting the mean instead of sorting to find the median.

---

3. Comparison of Smoothing Filters

Smoothing Filter	Mathematical Type	Edge Preservation	Optimal Noise Target	Visual Effect
Average (Box)	Linear (Equal weights) [83]	Poor (Heavy blurring) [292]	Gaussian noise	Blurs details, checkerboard borders [39]
Weighted Average	Linear (Distance weights) [125]	Moderate [125]	Gaussian noise	Smooth blur with center-focus details
Gaussian Filter	Linear (Separable bell curve) [10]	Moderate	Gaussian noise [235]	Smooth blur, **no ringing artifacts** [60]
Median Filter	Non-linear (Sorting-based) [20]	Excellent (Sharp edges preserved) [292]	Salt & Pepper noise [247]	Noise removed, fine lines preserved [292]

📝 SECTION 5: QUICK REVISION SHEET

1. Point Processing Formula Directory

- **Digital Negative:** $s = (L-1) - r$ [66]
- **Piecewise Linear Slopes:**

  $$\alpha = \frac{s_1}{r_1}, \quad \beta = \frac{s_2 - s_1}{r_2 - r_1}, \quad \gamma = \frac{(L-1) - s_2}{(L-1) - r_2}$$

  [281]
- **Log Transform:** $s = c \log_{10}(1 + r)$ [41, 65, 256]
- **Power-Law Transform:** $s = c \cdot r^{\gamma}$ [43, 65, 231]
- **Histogram Equalization:** $s_k = \text{round}\left((L-1) \cdot \sum_{j=0}^{k} \frac{n_j}{N}\right)$

⚠️ OUT OF MID-SEM SYLLABUS — DO NOT STUDY

The following topics are not included in the Mid-Sem syllabus but are kept here strictly for reference to align with the complete Unit 2 presentation material:

1. Sharpening Filters (High-Pass / Derivatives)

- **Basic Concept:** Highlights transitions in intensity levels, enhancing edges and fine structural details [174].
- **The Laplacian Operator:** A 2-D second-order isotropic derivative operator [116].

  $$\text{Standard Laplacian Mask} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Step-by-Step Sharpening Numerical (LMS Notes):** For target pixel $(2,2)$ in:

  $$I = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 40 & 30 & 20 & 10 \\ 10 & 20 & 30 & 40 \\ 40 & 30 & 20 & 10 \end{bmatrix}$$

  Using a Laplacian filter having a **positive center without diagonal elements**:

  $$H = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \implies g(2,2) = 20(-1) + 40(-1) + 20(-1) + 20(-1) + 30(4) = \mathbf{0}$$